
Chapitre – Conception

Conception Detaillee

Architecture, Diagrammes UML, Modele de Donnees et Algorithmes IA/ML

Realise par :

Soubaili Omar Yassine Baizi

Encadre par :

Mme. OUNACER SOUMAYA

Filiere : 4IADM G5-A

Annee Universitaire : 2025–2026

Table des matières

1	Introduction	3
2	Architecture Generale	3
2.1	Architecture 3-Tiers	3
2.2	Explication Detaillee des Couches	4
2.2.1	Couche Presentation (Frontend)	4
2.2.2	Couche Metier (Backend)	5
2.2.3	Couche Donnees	5
3	Diagramme de Cas d'Utilisation	5
3.1	Representation Graphique	6
3.2	Description des Acteurs	6
3.3	Detail des Cas d'Utilisation	7
3.3.1	Utilisateur (Standard) – UC01 a UC08	7
3.3.2	Manager (en plus des UC01–UC08)	8
3.3.3	Administrateur (en plus des UC01–UC14)	8
4	Diagramme de Classes	9
4.1	Representation Graphique	9
4.2	Explication des Classes	9
4.3	Relations entre Classes	10
5	Diagrammes de Sequence	11
5.1	Sequence : Authentification (Login)	12
5.2	Sequence : Recommandation d'Expert	13
5.3	Sequence : PageRank des Experts	14
6	Modele de Donnees (Graphe Neo4j)	15
6.1	Schema du Graphe	15
6.2	Proprietes des Noeuds	16
6.3	Comparaison Neo4j vs SQL	16
7	Conception des Algorithmes d'IA/ML	16
7.1	Tableau Recapitulatif	17
7.2	Filtrage Collaboratif (Prediction de Lacunes)	17
7.3	Detection des Competences Emergentes	17
7.4	Collaboration Inter-Departement	18
8	Conception de la Securite	18
8.1	Flux d'Authentification	18
8.2	Controle d'Acces (RBAC)	18
9	Conclusion	19

1. Introduction

Ce chapitre presente la conception detaillee de la plateforme **Nexora**. Il couvre l'architecture logicielle, les diagrammes UML (cas d'utilisation, classes, sequences), le modele de donnees en graphe Neo4j, ainsi que la conception mathematique des algorithmes d'Intelligence Artificielle et de Machine Learning. L'objectif est de fournir une vue technique complete et argumentee du fonctionnement interne du systeme.

La conception de Nexora repose sur trois principes fondateurs :

1. **Separation des responsabilites** : chaque couche de l'architecture a un role precis et bien delimite.
2. **Modelisation en graphe** : les entites et leurs relations sont naturellement structurees sous forme de graphe, exploitant les forces de Neo4j.
3. **Intelligence integree** : les algorithmes ML sont concus pour fonctionner en temps reel, directement embarques dans l'API backend.

2. Architecture Generale

2.1 Architecture 3-Tiers

L'application Nexora adopte une architecture **3-tiers** classique, separant strictement la presentation, la logique metier et les donnees. Ce choix architectural garantit la main-tenabilite, la scalabilite et la testabilite independante de chaque couche.

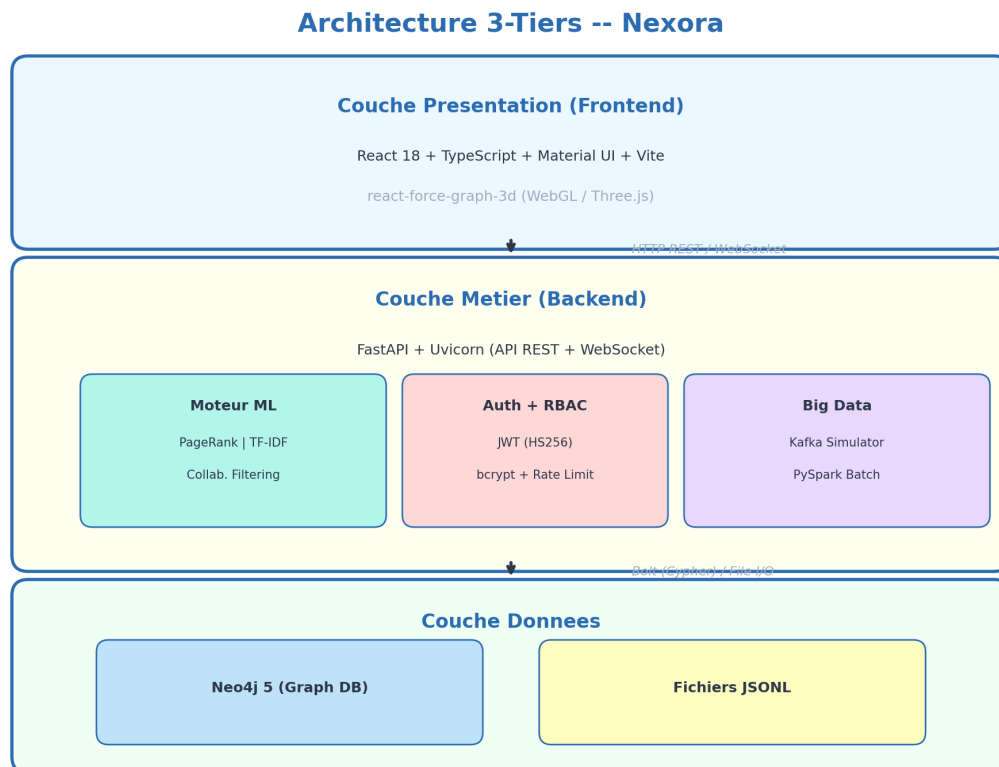


FIGURE 1 – Architecture 3-Tiers de Nexora

2.2 Explication Detaillee des Couches

2.2.1 Couche Presentation (Frontend)

La couche presentation est une application **SPA** (Single Page Application) construite avec React 18 et TypeScript. Elle est servie par Nginx en production et communique avec le backend exclusivement via des requetes REST (bibliotheque Axios) et des connexions WebSocket (chat temps reel).

Justification des choix :

- **React 18** : Son architecture a base de composants et son Virtual DOM permettent un rendu performant, essentiel pour l’affichage du graphe interactif.
- **TypeScript** : Le typage statique reduit les bugs et ameliore la maintenabilite du code frontend.
- **Material UI** : Fournit un systeme de design complet et coherent (theming, responsive, accessibilite) sans surcharge de developpement.
- **react-force-graph-3d** : Permet le rendu 3D du graphe organisationnel via WebGL/Three.js, offrant une experience immersive a 60 FPS.
- **Vite** : Bundler de nouvelle generation offrant un HMR (Hot Module Replacement) quasi-instantane pendant le developpement.

2.2.2 Couche Metier (Backend)

Le backend centralise toute la **logique metier** de l'application : authentification, controle d'accès, moteur de recommandation ML, pipelines Big Data et gestion des données.

Justification des choix :

- **FastAPI** : Framework Python asynchrone qui genere automatiquement la documentation OpenAPI/Swagger. Ses performances sont proches de Node.js grace a l'ASGI.
- **Uvicorn** : Serveur ASGI haute performance capable de gerer simultanement les connexions HTTP REST et WebSocket.
- **Pydantic v2** : Validation des donnees avec schemas types, garantissant l'integrite des echanges entre le frontend et le backend.
- **Scikit-learn** : Librairie ML mature pour le TF-IDF, la regression logistique et le calcul de similarite cosinus.

2.2.3 Couche Donnees

La couche donnees combine une base **Neo4j** orientee graphe et des fichiers **JSONL** comme source de donnees pour les modeles ML.

Justification des choix :

- **Neo4j 5** : Base de donnees native orientee graphe avec adjacence sans index (*index-free adjacency*), permettant des traversees en temps constant $O(1)$ par relation.
- **Langage Cypher** : Langage declaratif intuitif pour exprimer des motifs de graphe complexes.
- **JSONL** : Format leger et streamable, compatible nativement avec Pandas, PySpark et les pipelines ML.

3. Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation modelise les interactions entre les acteurs du systeme et les fonctionnalites offertes par Nexora. Il suit le modele d'**heritage de roles** : chaque role herite des droits du role inferieur.

3.1 Representation Graphique

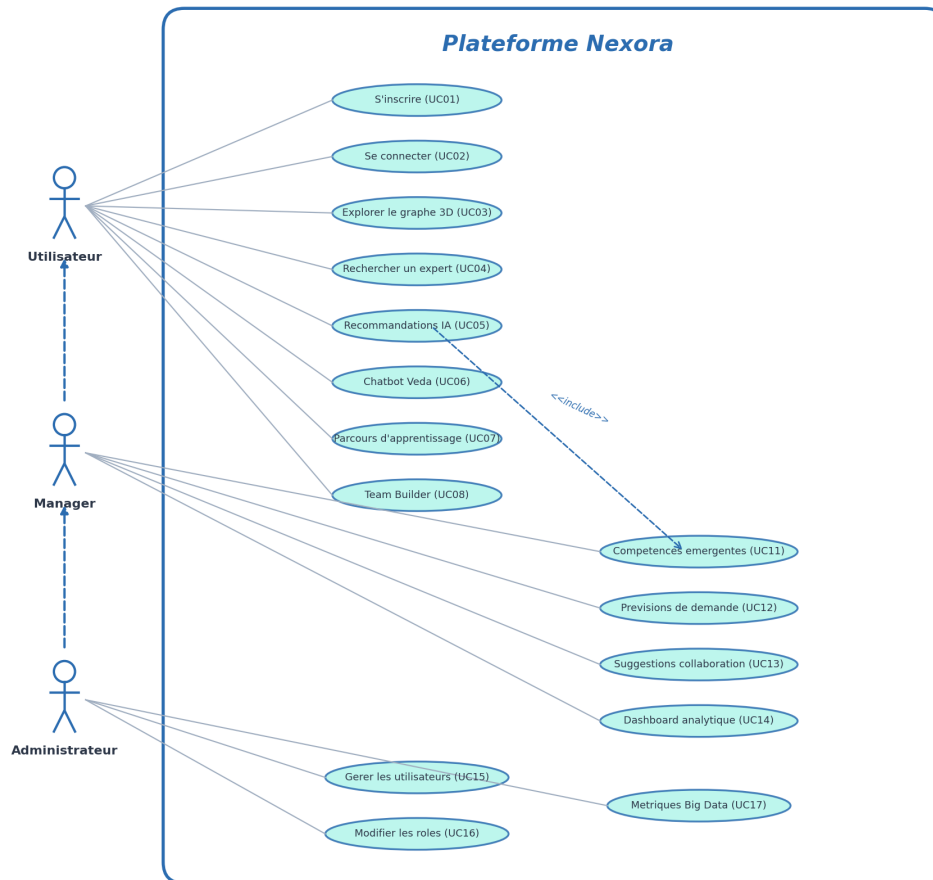


FIGURE 2 – Diagramme de Cas d'Utilisation – Nexora

3.2 Description des Acteurs

Acteur	Description et Responsabilités
Utilisateur (User)	Employé standard de l'organisation. Il représente le profil de base ayant accès aux fonctionnalités courantes : exploration du graphe, recherche d'experts, recommandations IA, chatbot, et outils de collaboration.
Manager	Hérite de tous les droits de l'Utilisateur. En plus, il accède aux outils d'analyse stratégique : prévisions de compétences, compétences émergentes, suggestions de collaboration inter-département, et dashboard analytique. Ce rôle correspond aux responsables de département ou chefs d'équipe.

Administrateur	Herite de tous les droits du Manager. Il dispose en plus des droits d'administration systeme : gestion des utilisateurs (CRUD), modification des roles, acces aux metriques Big Data (pipeline Kafka, rapports Spark), et supervision technique de la plateforme.
----------------	---

Explication du modele : Ce modele d'heritage a trois niveaux est implemente via le systeme RBAC (Role-Based Access Control) avec des tokens JWT. Chaque endpoint de l'API verifie le role minimum requis avant d'autoriser l'acces. Par exemple, un endpoint marque `require_role("manager")` est accessible aux Managers et aux Administrateurs, mais interdit aux Utilisateurs standards.

3.3 Detail des Cas d'Utilisation

3.3.1 Utilisateur (Standard) – UC01 a UC08

ID	Cas d'utilisation	Description et Flux
UC01	S'inscrire	L'utilisateur remplit un formulaire (username, email, mot de passe). Le backend hache le mot de passe avec bcrypt (12 rounds) et cree un noeud User dans la base. Le role par default est "user".
UC02	Se connecter	L'utilisateur soumet ses identifiants. Le backend verifie le hash bcrypt, controle le rate limiting (5 tentatives / 5 min), puis genere un JWT signe (HS256) contenant le username et le role, valide 24h.
UC03	Explorer le graphe 3D	Le frontend charge les donnees du graphe (noeuds + aretes) via <code>GET /api/graph</code> et les rend en 3D avec Three.js. L'utilisateur peut zoomer, tourner, filtrer par type de noeud et cliquer pour voir les details.
UC04	Rechercher un expert	L'utilisateur saisit une requete libre (ex : "Python security Paris"). Le backend vectorise la requete avec TF-IDF et calcule la similarite cosinus avec tous les profils. Les resultats sont tries par score.
UC05	Recommandations IA	Le moteur IA analyse le profil de l'utilisateur connecte et suggere des experts similaires ou complementaires via le filtrage collaboratif sur la matrice User-Skill.
UC06	Chatbot Veda	L'utilisateur interagit en langage naturel avec le chatbot IA. Veda utilise un LLM enrichi par le contexte du graphe organisationnel (Graph-RAG) pour fournir des reponses contextuelles.

UC07	Parcours d'apprentissage	Le systeme identifie les lacunes de competences de l'utilisateur par filtrage collaboratif et suggere un parcours d'apprentissage personnalise.
UC08	Team Builder	Le Team Builder permet de specifier les competences requises pour un projet. L'algorithme selectionne une equipe optimale en maximisant la couverture des skills.

3.3.2 Manager (en plus des UC01–UC08)

ID	Cas d'utilisation	Description et Flux
UC11	Competences emergentes	L'algorithme d'analyse de cohortes compare les taux d'adoption des competences entre les recrues recentes (< 2 ans) et les employes plus anciens (2–5 ans). Les competences dont l'adoption est en forte hausse sont marquees comme "emergentes".
UC12	Previsions de demande	Le systeme croise la demande actuelle des projets avec les taux de croissance des competences pour projeter les besoins a 6–24 mois. Les lacunes critiques (gap offre/demande) sont mises en evidence.
UC13	Suggestions collaboration	L'analyse ensembliste compare les profils de competences de chaque departement. Les paires ayant une forte complementarite sont suggerees pour la collaboration.
UC14	Dashboard analytique	Le dashboard affiche des KPIs globaux : nombre d'experts, repartition des competences, projets actifs, tendances, et statistiques de la plateforme.

3.3.3 Administrateur (en plus des UC01–UC14)

ID	Cas d'utilisation	Description et Flux
UC15	Gerer les utilisateurs	L'administrateur peut lister tous les comptes, voir les details, et superviser l'activite. L'endpoint <code>GET /api/auth/users</code> est reserve au role "admin".
UC16	Modifier les roles	L'administrateur peut promouvoir ou retrograder un utilisateur via <code>PUT /api/auth/users/{id}/role</code> . Ce changement est immediatement reflecte dans les tokens JWT suivants.

UC17	Metriques Big Data	L'administrateur accede au statut du pipeline Kafka (events/sec, queue depth, erreurs) et aux resultats des analyses Spark (skill analytics, expert rankings).
------	--------------------	--

4. Diagramme de Classes

Le diagramme de classes modelise les entites metier principales de Nexora et leurs relations. Dans le contexte d'une base de donnees orientee graphe (Neo4j), les classes correspondent aux **noeuds** et les associations aux **relations** (aretes).

4.1 Representation Graphique

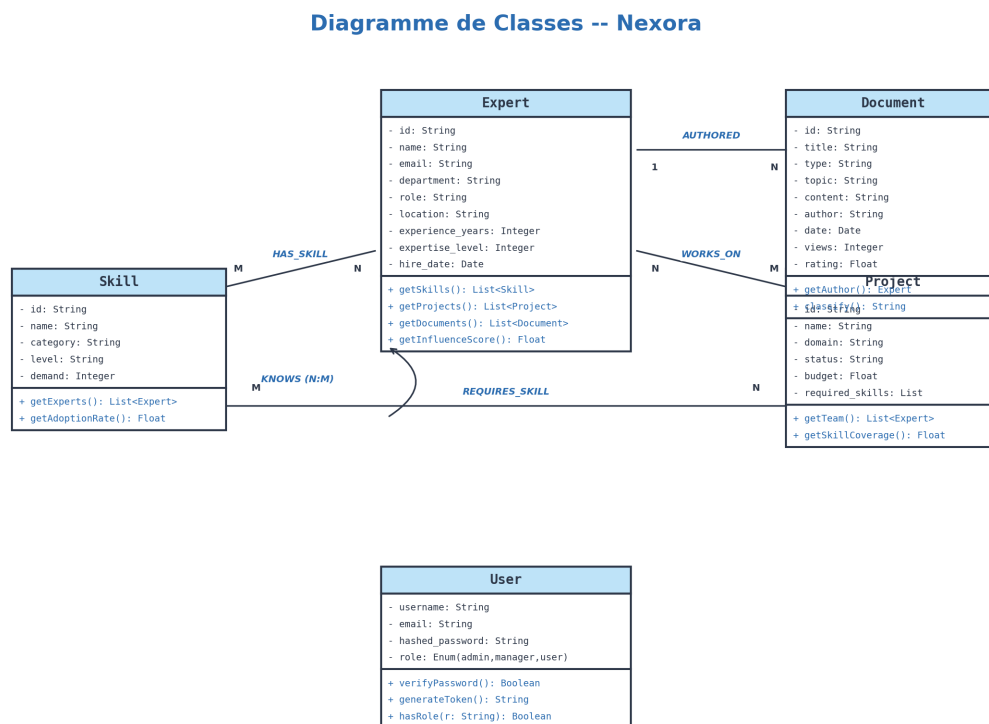


FIGURE 3 – Diagramme de Classes – Nexora

4.2 Explication des Classes

Classe	Role et Attributs Cles
--------	------------------------

Expert	Entite centrale du graphe representant un employe. L'attribut experience_years est un facteur du score PageRank. Le champ hire_date est exploite par l'algorithme de detection des competences emergentes pour construire les cohortes temporelles.
Skill	Competence technique referencee. Le champ category regroupe les 82 competences en 6 familles. Le champ demand (0–100) reflete la demande du marche, utilise dans les previsions futures.
Project	Projet technique de l'organisation. La relation WORKS_ON entre Expert et Project est essentielle pour le PageRank : un expert travaillant sur de nombreux projets critiques aura un score d'influence plus eleve.
Document	Base documentaire interne. Le contenu est vectorise par TF-IDF pour la classification automatique. Le nombre de documents publies contribue au score PageRank (facteur $\times 2.5$).
User	Classe d'authentification distincte de Expert. Le mot de passe est stocke sous forme de hash bcrypt (12 rounds). La methode generateToken() cree un JWT signe HS256.

4.3 Relations entre Classes

Relation	Type Neo4j	Card.	Explication
Expert $\rightarrow$ Skill	HAS_SKILL	N :M	Un expert possede plusieurs competences, et une competence est maitrisee par plusieurs experts. La relation porte un attribut level (1–5).
Expert $\rightarrow$ Project	WORKS_ON	N :M	Un expert participe a plusieurs projets, et un projet a une equipe de 3–12 membres. Cle du graphe biparti PageRank.
Expert $\rightarrow$ Document	AUTHORED	1 :N	Un document a un seul auteur, mais un expert peut publier plusieurs documents. Contribue au score d'influence.
Expert $\rightarrow$ Expert	KNOWS	N :M	Relation de collaboration deduite automatiquement quand deux experts partagent un projet ou des competences.
Project $\rightarrow$ Skill	REQUIRES_SKILL	N :M	Un projet necessite certaines competences. Permet de calculer le “skill coverage” de l'equipe.

Document $\rightarrow$ Topic	COVERS_TOPIC	N:M	Un document traite un ou plusieurs sujets. Utilise pour la classification et la navigation thematique.
------------------------------	--------------	-----	--

Pourquoi Neo4j plutôt que SQL ? Dans une base relationnelle, trouver “tous les experts qui partagent au moins 3 compétences avec l’expert X” nécessiterait 4–5 jointures imbriquées. En Neo4j, cette requête s’exprime naturellement en Cypher :

```

1 MATCH (e1:Expert {id: $expert_id}) -[:HAS_SKILL] ->(s:Skill)
2     <-[:HAS_SKILL] -(e2:Expert)
3 WHERE e1 <> e2
4 WITH e2, COUNT(s) AS shared_skills
5 WHERE shared_skills >= 3
6 RETURN e2.name, shared_skills
7 ORDER BY shared_skills DESC

```

Le temps d’exécution est proportionnel au nombre de relations traversées, pas à la taille totale de la base ($O(k)$ au lieu de $O(n^2)$ pour les jointures SQL).

5. Diagrammes de Sequence

Les diagrammes de sequence détaillent le flux d’exécution des processus clés du système, montrant les interactions temporelles entre les composants.

5.1 Sequence : Authentication (Login)

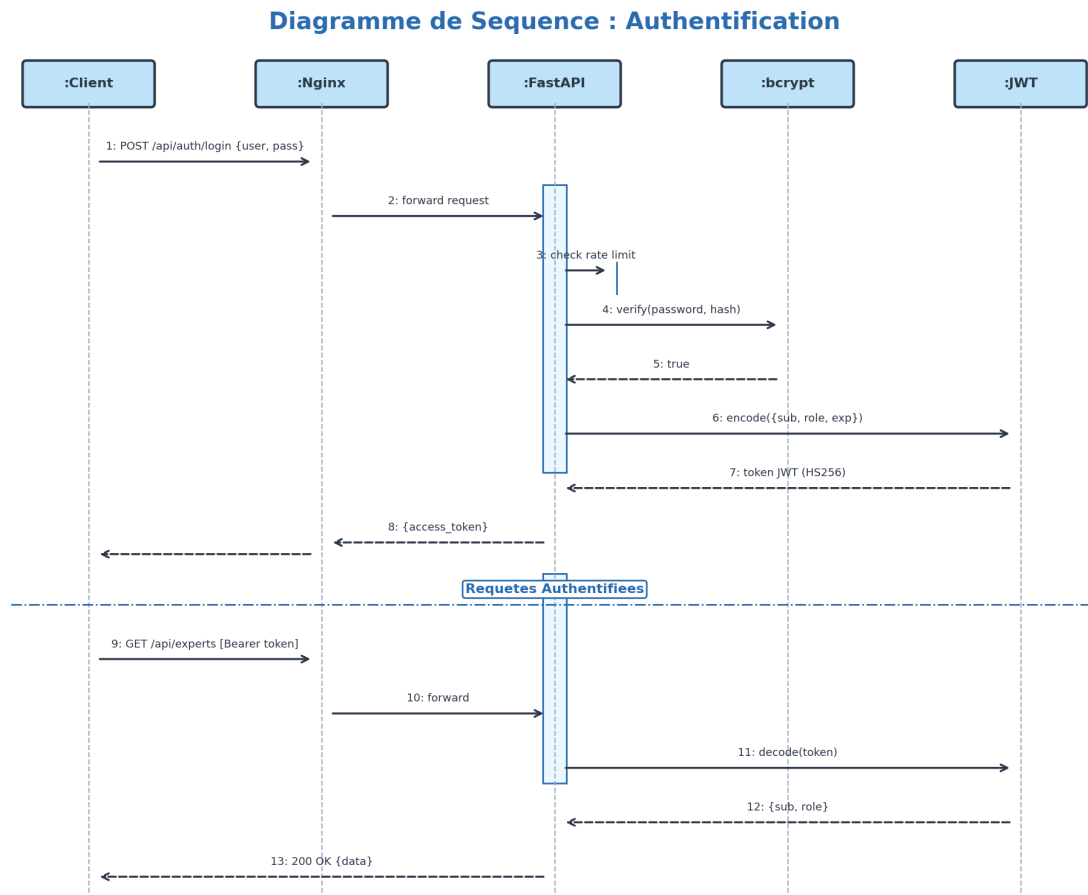


FIGURE 4 – Diagramme de Sequence – Authentication JWT

Explication detaillee :

- Envoi des identifiants :** Le client (navigateur React) envoie une requete POST `/api/auth/login` avec un body JSON contenant `username` et `password`.
- Proxy Nginx :** Nginx recoit la requete sur le port 3000 et la redirige vers le backend FastAPI (port 8000) grace a la directive `proxy_pass`.
- Rate Limiting :** Avant toute verification, le systeme controle que l'utilisateur n'a pas depasse 5 tentatives echouees dans les 5 dernieres minutes. Ce mecanisme protege contre les attaques par dictionnaire.
- Verification bcrypt :** FastAPI utilise `bcrypt.verify(password, hashed_password)` pour comparer. Le bcrypt utilise 12 rounds de salage, rendant les attaques par force brute impossibles (~250ms par tentative).
- Generation JWT :** Si le mot de passe est valide, un JWT est signe avec HS256 : `{sub: "username", role: "admin", exp: now+24h}`. Le token est renvoye au client.
- Requetes authentifiees :** Pour les requetes suivantes, le client inclut le token dans le header `Authorization: Bearer <token>`. FastAPI decode le token et verifie le role avant de repondre.

5.2 Sequence : Recommandation d'Expert

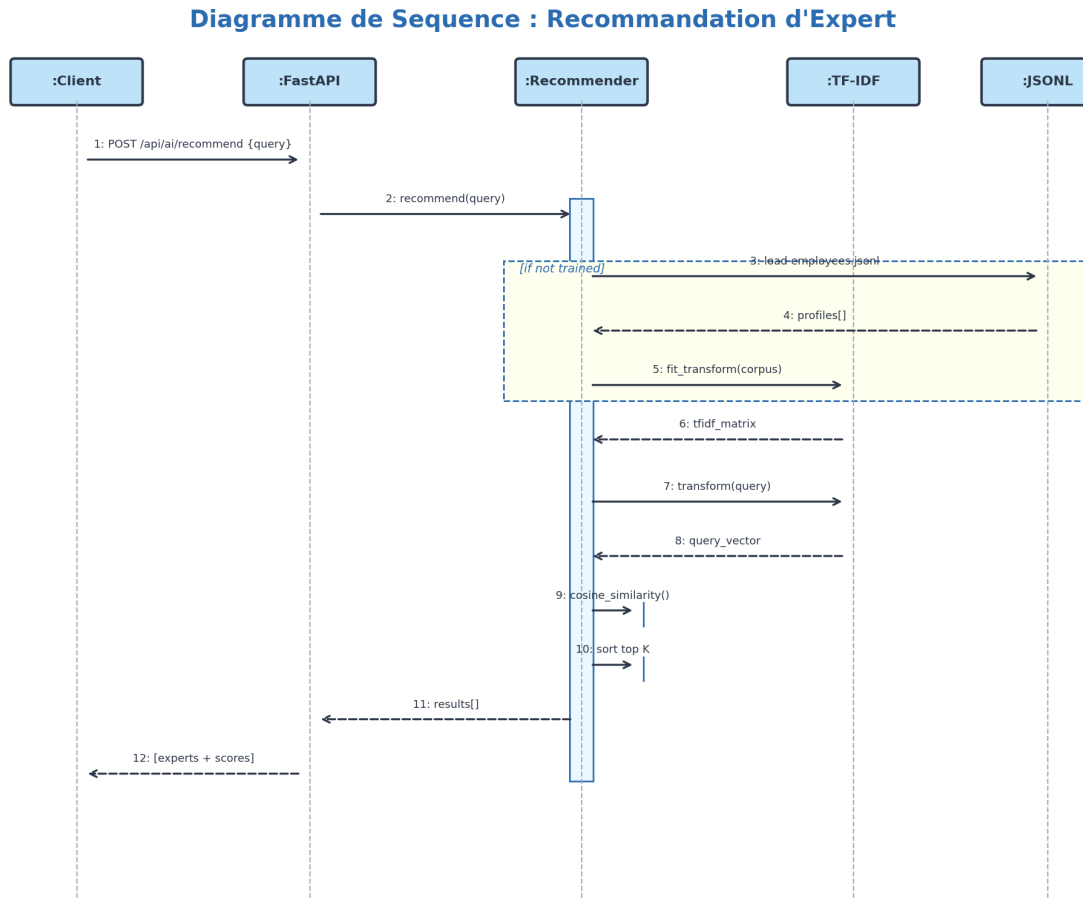


FIGURE 5 – Diagramme de Sequence – Recommandation d'Expert (TF-IDF)

Explication detaillee de l'algorithme :

1. **Requete utilisateur** : Le client envoie une requete textuelle libre decrivant le besoin (ex : “expert Python machine learning Paris”).
2. **Chargement lazy** : Si le modele n'est pas encore entraine, le Recommender charge les profils experts depuis les fichiers JSONL (pattern Singleton, une seule fois).
3. **Construction du corpus** : Pour chaque expert, un texte de profil est construit en concatenant : nom, role, departement, localisation et noms de competences repetes proportionnellement a leur niveau (une competence de niveau 4 apparait 4 fois), simulant une ponderation naturelle.
4. **Vectorisation TF-IDF** : Les profils textuels sont transformes en vecteurs numeriques sparse (5000 dimensions max, n-grammes 1–2) :

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \frac{N}{\text{DF}(t)} \quad (1)$$

ou $\text{TF}(t, d)$ est la frequence du terme t dans le document d , N est le nombre total de documents, et $\text{DF}(t)$ est le nombre de documents contenant t .

5. **Similarite cosinus** : La requete est transformee dans le meme espace vectoriel, puis la similarite cosinus est calculee :

$$\cos(\theta) = \frac{\vec{q} \cdot \vec{p}}{||\vec{q}|| \times ||\vec{p}||} \quad (2)$$

Un score de 1.0 indique une correspondance parfaite, 0.0 une absence totale de similarite.

6. **Tri et retour** : Les K experts avec les scores les plus eleves sont retournees avec leurs profils et scores.

5.3 Sequence : PageRank des Experts

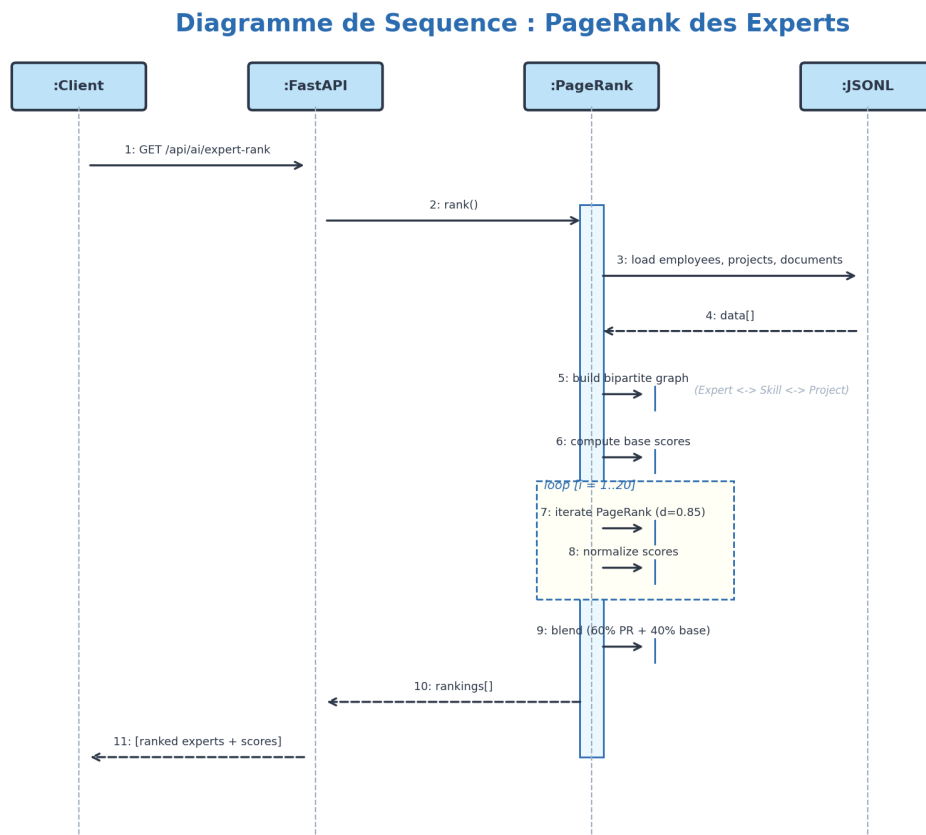


FIGURE 6 – Diagramme de Sequence – PageRank des Experts

Explication detaillee de l'algorithme PageRank :

1. **Construction du graphe biparti** : Un graphe non-oriente est construit avec trois types de noeuds : experts (`emp_*`), competences (`_sk_*`), et projets (`proj_*`). Les aretes connectent les experts a leurs competences et projets.
2. **Score de base** : Pour chaque expert, un score brut est calcule :

$$S_{\text{base}}(e) = 1.5 \times B + 2.0 \times D + 3.0 \times P + 2.5 \times C + 1.0 \times \ln(1 + E) \quad (3)$$

ou B = nombre de competences (breadth), D = niveau moyen (depth), P = nombre de projets, C = nombre de documents, et E = annees d'experience.

3. **Iteration PageRank** : L'algorithme itere 20 fois avec un facteur d'amortissement $d = 0.85$ (valeur standard de Brin & Page, 1998) :

$$PR(e_i) = \frac{1-d}{N} + d \sum_{e_j \in \text{voisins}(e_i)} \frac{PR(e_j)}{|\text{sortants}(e_j)|} \quad (4)$$

Les “voisins” sont les experts connectés via des compétences ou projets partagés (voisins à 2 sauts dans le graphe biparti).

4. **Normalisation** : Apres chaque iteration, les scores sont normalises pour que leur somme soit egale a 100.
5. **Blending final** : Le score final combine le PageRank (60%) avec le score de base normalise (40%) :

$$S_{\text{final}}(e) = 0.6 \times PR(e) + 0.4 \times S_{\text{base_norm}}(e) \quad (5)$$

Cette combinaison evite que des experts isolés mais très expérimentés soient sous-évalués par le PageRank pur.

6. Modele de Donnees (Graphe Neo4j)

6.1 Schema du Graphe

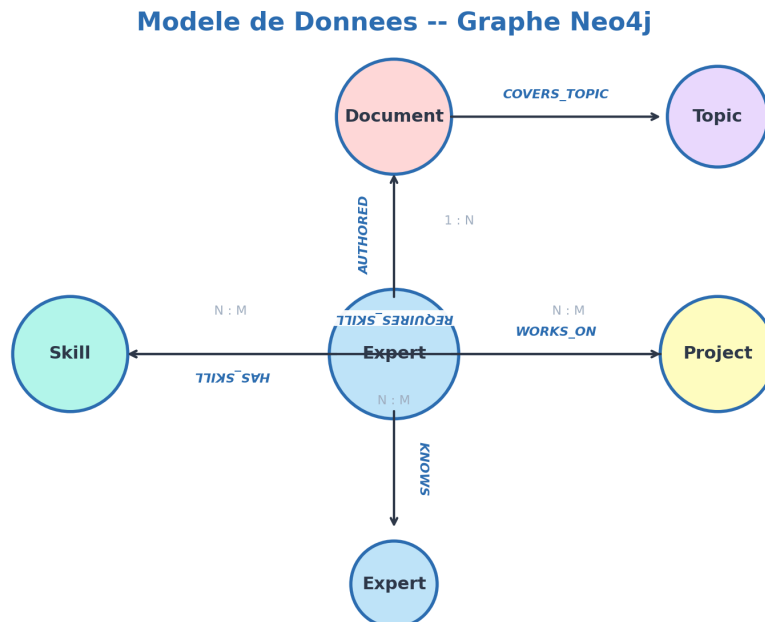


FIGURE 7 – Modele de Donnees – Graphe Neo4j

Explication du schema :

- **Expert (noeud central)** : C’est le noeud le plus connecte du graphe. Il possede des aretes vers Skill, Project, Document et d’autres Expert. Cette centralite est exactement ce que le PageRank exploite.
- **Skill** : Les competences sont partagees entre experts (HAS_SKILL) et requises par des projets (REQUIRES_SKILL). Ce double lien cree des chemins indirects Expert $\rightarrow$ Skill $\leftarrow$ Project, exploites par les algorithmes de recommandation.
- **Project** : Les projets relient les experts qui travaillent ensemble (WORKS_ON). Deux experts sur le meme projet forment implicitement une relation KNOWS.
- **Document** : Les documents publies (AUTHORED) contribuent au score d’influence de l’expert. Leur contenu est utilise pour la classification automatique.
- **Topic** : Les sujets permettent de categoriser les documents et d’offrir une navigation thematique.

6.2 Proprietes des Noeuds

Noeud	Proprietes et Types
Expert	id (String), name (String), email (String), department (String), role (String), location (String), experience_years (Integer), expertise_level (Integer 1–5), hire_date (Date ISO)
Skill	id (String), name (String), category (String parmi 6 categories), level (String : Beginner/Intermediate/Advanced), demand (Integer 0–100)
Project	id (String), name (String), domain (String parmi 20 domaines), status (String), priority (String), budget (Float), tech_stack (List), required_skills (List)
Document	id (String), title (String), type (String parmi 9 types), topic (String), content (String), author (String $\rightarrow$ Expert.id), date (Date), views (Integer), rating (Float)

6.3 Comparaison Neo4j vs SQL

Critere	Neo4j (Graphe)	SQL (Relationnel)
Traversee de relations	$O(k)$ – constant par relation	$O(n \log n)$ – jointures
Requetes multi-sauts	Natif (Cypher MATCH)	Sous-requetes imbriquees
Algorithmes de graphe	PageRank, centralite natifs	Calculs externes requis
Schema	Flexible, sans migration	Rigide, ALTER TABLE
Echelle relationnelle	100K+ noeuds, millions de relations	Degrade avec les jointures

7. Conception des Algorithmes d’IA/ML

Nexora integre un pipeline ML complet avec 7 algorithmes complementaires, chacun concu pour resoudre un aspect specifique de l’intelligence organisationnelle.

7.1 Tableau Recapitulatif

Algorithme	Technique	Entree	Sortie
Recommandation d'experts	TF-IDF + similarite cosinus	Requete textuelle libre	Top K experts tries par score
Scoring d'influence	PageRank iteratif (d=0.85, 20 iter.)	Graphe biparti Expert-Skill-Project	Score d'influence global
Prediction de lacunes	Filtrage collaboratif	Matrice User-Skill	Competences recommandees
Competences emergentes	Analyse de cohortes temporelles	Profils + dates d'embauche	Competences en hausse/declin
Prevision de demande	Projection lineaire	Projets + taux de croissance	Demande a 6–24 mois
Classification de docs	TF-IDF + Regression Logistique	Contenu textuel des documents	Categorie predite
Collaboration inter-dept.	Analyse ensembliste	Competences par departement	Paires complementaires

7.2 Filtrage Collaboratif (Prediction de Lacunes)

1. **Construction de la matrice User-Skill** : Une matrice M de taille $(n_users \times n_skills)$ est construite, ou $M[u][s]$ contient le niveau de maitrise (1–5) de l'utilisateur u pour la competence s , ou 0 si absente.
2. **Similarite entre experts** : La similarite cosinus est calculee entre le vecteur de l'expert cible et ceux de tous les autres experts.
3. **Aggregation des lacunes** : Pour chaque competence que l'expert cible ne possede pas :

$$\text{score}(s) = \sum_{e \in \text{top-10 voisins}} \text{sim}(e_{\text{cible}}, e) \times M[e][s] \quad (6)$$

Les competences sont trieées par score decroissant et les top-K sont retournees comme recommandations de formation.

7.3 Detection des Competences Emergentes

L'algorithme utilise une **analyse de cohortes** basee sur les dates d'embauche :

1. **Cohorte recente** : Employes embauches dans les 2 dernieres annees.
2. **Cohorte ancienne** : Employes embauches entre 2 et 5 ans.
3. **Taux d'adoption par cohorte** :

$$R_{\text{recent}} = \frac{\text{Nb experts recents avec skill}}{\text{Nb total experts recents}}, \quad R_{\text{ancien}} = \frac{\text{Nb experts anciens avec skill}}{\text{Nb total experts anciens}} \quad (7)$$

4. **Taux de croissance :**

$$\text{growth} = \frac{R_{\text{recent}} - R_{\text{ancien}}}{\max(R_{\text{ancien}}, 0.01)} \quad (8)$$

5. **Classification :** $\text{growth} > 0.2$: “rising”. Entre -0.1 et 0.2 : “stable”. < -0.1 : “declining”.

7.4 Collaboration Inter-Departement

L’algorithme mesure la **complementarite** entre deux departements :

1. Pour chaque departement, l’ensemble de ses competences est construit : S_A, S_B .
2. **Competences partagees :** $\text{overlap} = |S_A \cap S_B|$
3. **Complementarite :** $\text{comp} = |S_A \setminus S_B| + |S_B \setminus S_A|$
4. **Score de potentiel :**

$$\text{potential} = \frac{\text{comp}}{|S_A \cup S_B|} \times 100 \quad (9)$$

Les paires avec $\text{comp} > 3$ et $\text{overlap} > 0$ sont retenues.

8. Conception de la Securite

8.1 Flux d’Authentification

Etape	Description et Mecanisme
1	L’utilisateur saisit son username et mot de passe sur la page de login React.
2	Le frontend envoie <code>POST /api/auth/login</code> avec les identifiants en JSON. La connexion est chiffree via HTTPS (TLS 1.3).
3	Le backend verifie le rate limiting : maximum 5 tentatives echouees par IP sur une fenetre de 5 minutes. Au-dela, reponse HTTP 429.
4	Le mot de passe est compare au hash bcrypt stocke (12 rounds de salage, $\sim 250\text{ms}$ par verification).
5	Si valide : un token JWT est signe avec HS256. Payload : <code>{sub: "username", role: "admin", exp: now+24h}</code> .
6	Le token est stocke dans <code>localStorage</code> et envoye dans le header <code>Authorization: Bearer <token></code> pour chaque requete API.

8.2 Controle d’Acces (RBAC)

Fonctionnalite	User	Manager	Admin
Explorer le graphe 3D	✓	✓	✓
Rechercher des experts	✓	✓	✓
Recommandations IA	✓	✓	✓
Chatbot Veda	✓	✓	✓
Team Builder	✓	✓	✓

Parcours d'apprentissage	✓	✓	✓
Compétences émergentes		✓	✓
Prévisions compétences		✓	✓
Dashboard analytique		✓	✓
Gérer les utilisateurs			✓
Modifier les rôles			✓
Métriques Big Data			✓

Implementation : Chaque endpoint est décoré avec `require_role(min_role)`. La hiérarchie est : `user < manager < admin`.

9. Conclusion

La conception de Nexora repose sur une architecture modulaire et bien structurée, où chaque composant a un rôle clairement défini. Les diagrammes UML (cas d'utilisation, classes, séquences) formalisent les interactions entre les acteurs et les entités du système. Le modèle de données en graphe Neo4j exploite naturellement les relations complexes entre experts, compétences, projets et documents.

Les algorithmes d'IA/ML – PageRank adapté, TF-IDF + similarité cosinus, filtrage collaboratif, analyse de cohortes – sont conçus pour fonctionner en temps réel et s'alimenter directement des données JSONL. Leur fondement mathématique (équations TF-IDF, PageRank itératif, similarité cosinus, analyse ensembliste) garantit des résultats pertinents et explicables.

Cette conception constitue une base solide pour l'implémentation, assurant la cohérence entre les besoins fonctionnels identifiés dans le cahier des charges et la réalité technique du code produit.